Ejercicios sobre las implementaciones y quicksort

Pagar al partir para no pagar al combinar

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Resolver problemas sobre la lista, la pila y la cola enlazadas, y dar el costo en tiempo y en espacio de cada solución (OA2, OA3).
- Explicar cómo `particionar` deja el pivote en su lugar definitivo, con las dos colas que viven dentro del arreglo (OA1).
- Construir quicksort sobre `particionar`, con recursión y con una pila de rangos, y justificar su costo en tiempo y en espacio (OA1, OA2, OA3).
- Comparar insertion sort, merge sort y quicksort con comparaciones contadas sobre la misma entrada, y escoger uno según los datos (OA1, OA5).

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

La lista, la pila y la cola sobre nodos I

TAD	Operación	Tiempo	Por qué
Lista	`insertar(0, e)`	$\Theta(1)$	se escribe la cabeza
	`agregar(e)`	$\Theta(1)$	se cuelga de `ultimo`
	`obtener(p)` y similares	$\Theta(p)$	se camina hasta p
Pila	`apilar`, `desapilar`, `tope`	$\Theta(1)$	la cima es la cabeza
Cola	`encolar`	$\Theta(1)$	se cuelga de `ultimo`
	`desencolar`, `frente`	$\Theta(1)$	se sale por la cabeza
Las tres	`tamano`, `vacia`	$\Theta(1)$	se lee `n`
	espacio	$\Theta(n)$	dato y puntero por nodo

Ninguna tiene techo: cada elemento reserva su nodo al entrar y lo libera al salir. A cambio, llegar a una posición del medio exige caminar.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Ejercicios en parejas I

(a) La cola doble

Una cola que deja entrar y salir por los dos extremos: `insertarFrente`, `insertarFinal`, `eliminarFrente` y `eliminarFinal`. ¿Qué lista hace falta debajo para que las cuatro sean $\Theta(1)$? ¿Cuál de las cuatro falla con la lista simple que guarda `ultimo`?

(b) La cola sin contador

Si la cola de nodos no guardara `n`, ¿cómo responde `vacia()`? ¿Y `tamano()`? ¿Cuánto cuesta cada una?

Ejercicios en parejas II

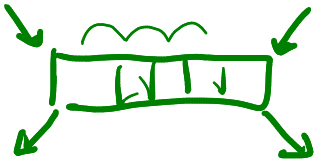
(c) Invertir una cola

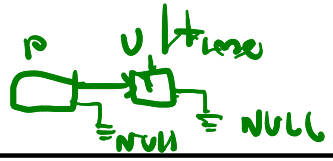
Déle la vuelta a una cola usando solo una pila auxiliar y las operaciones de los dos contratos: `encolar`, `desencolar`, `frente`, `vacia` de la cola y `apilar`, `desapilar`, `tope`, `vacia` de la pila. ¿Cuánto cuesta en tiempo? ¿Cuánto espacio pide aparte de la cola?

Las tres se escriben sobre los TAD que ya están construidos: la cola y la pila sobre nodos.

(a) La cola doble: dónde falla la lista simple I

Con la simple y `ultimo`, tres de las cuatro operaciones son $\Theta(1)$. La cuarta no.





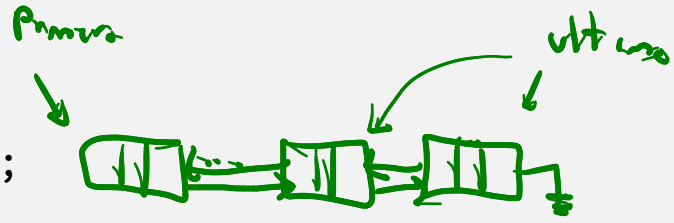
Operación	Simple con `ultimo`	Por qué
`insertarFrente`	$\Theta(1)$	se escribe la cabeza
`insertarFinal`	$\Theta(1)$	se cuelga de `ultimo`
`eliminarFrente`	$\Theta(1)$	la cabeza pasa al siguiente
`eliminarFinal`	$\Theta(n)$	hay que llegar al penúltimo

Quitar el último deja al penúltimo como nuevo último, y el penúltimo solo se alcanza caminando desde la cabeza. `ultimo` sirve para poner al final, no para quitar de ahí.

(a) La cola doble: dónde falla la lista simple II

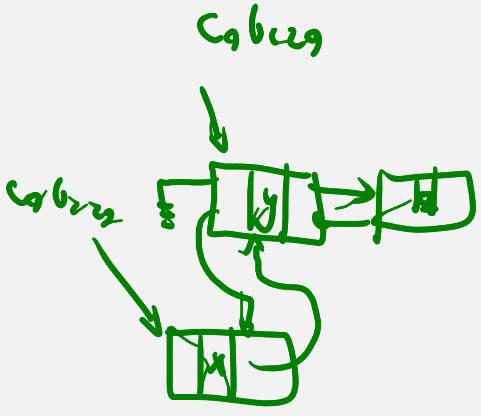
La lista doblemente enlazada con cabeza y `ultimo` resuelve la cuarta: el penúltimo es `ultimo->anterior`.

```
// exige !vacia()
void eliminarFinal() {
  assert(!vacia());
  NodoDoble *muerto = ultimo;
  ultimo = ultimo->anterior;
  if (ultimo == NULL) {
    cabeza = NULL;
  } else {
    ultimo->siguiente = NULL;
  }
  delete muerto;
  n = n - 1;
}
```



(a) La cola doble: dónde falla la lista simple III

```
void insertarFrente(Elemento e) {
  NodoDoble *nuevo = new NodoDoble;
  nuevo->dato = e;
  nuevo->anterior = NULL;
  nuevo->siguiente = cabeza;
  if (cabeza == NULL) {
    ultimo = nuevo;
  } else {
    cabeza->anterior = nuevo;
  }
  cabeza = nuevo;
  n = n + 1;
}
```



Las otras dos son el espejo de estas: se cambia `cabeza` por `ultimo` y `siguiente` por `anterior`.

Espacio que ocupan listas enlazadas de enteros

La lista simple

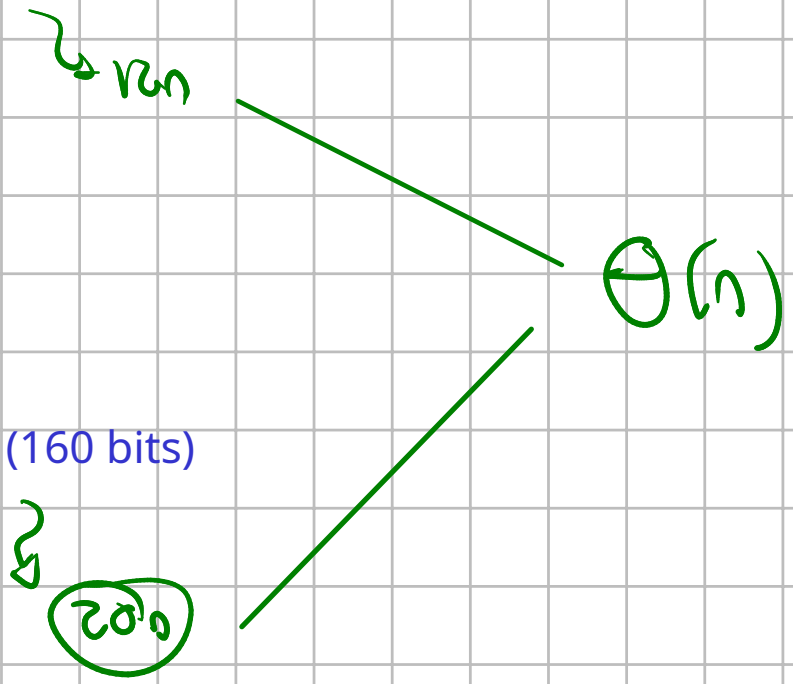
Nodo siguiente = 8 bytes (64 bits)
int dato = 4 bytes (32 bits)

Por nodo requiere 12 bytes. (96 bits)

Lista doblemente enlazada

Nodo anterior: 8 bytes
int dato = 4 bytes
Nodo siguiente: 8 bytes

Por nodo requiere 20 bytes (160 bits)



(a) La cola doble: dónde falla la lista simple IV



Una corrida

`insertarFinal(4)`, `insertarFinal(7)`, `insertarFrente(1)`: `frente 1, final 7, tamano 3.`
`eliminarFinal()`: `frente 1, final 4.`
`eliminarFrente()` y `eliminarFinal()`: la cola queda vacía, y `insertarFrente(9)` deja `frente 9, final 9`.

Operación	Doble con `ultimo`	Por qué
las cuatro de los extremos	$\Theta(1)$	a lo sumo cuatro punteros
`frente`, `final`	$\Theta(1)$	se lee `cabeza` o `ultimo`
espacio	$\Theta(n)$	dato y dos punteros por nodo

(a) La cola doble: dónde falla la lista simple V

El caso que rompe la cola doble

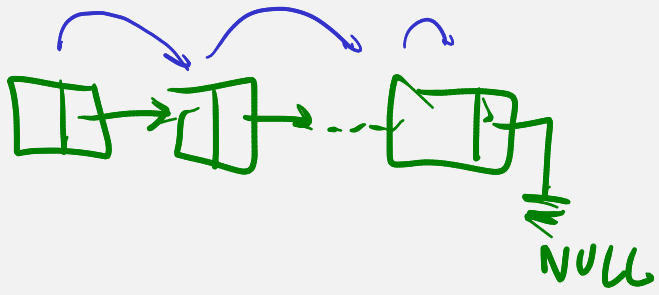
Al sacar el único nodo por cualquiera de los dos lados, el otro extremo también debe quedar en `NULL`. Si no, apunta a un nodo liberado.

(b) La cola sin contador I

Sin `n`, la cadena misma dice si hay elementos: la cola está vacía cuando no hay cabeza. Contarlos exige recorrerla.

```
bool vacia() {
  return cabeza == NULL;
}

int tamano() {
  int cuenta = 0;
  Nodo *actual = cabeza;
  while (actual != NULL) {
    cuenta = cuenta + 1;
    actual = actual->siguiente;
  }
  return cuenta;
}
```



(b) La cola sin contador II

Operación	Sin `n`	Por qué
`vacia()`	$\Theta(1)$	compara un puntero
`tamano()`	$\Theta(n)$	visita cada nodo una vez
`encolar`, `desencolar`, `frente`	$\Theta(1)$	no usaban `n`
espacio de la clase	dos punteros	uno menos que antes

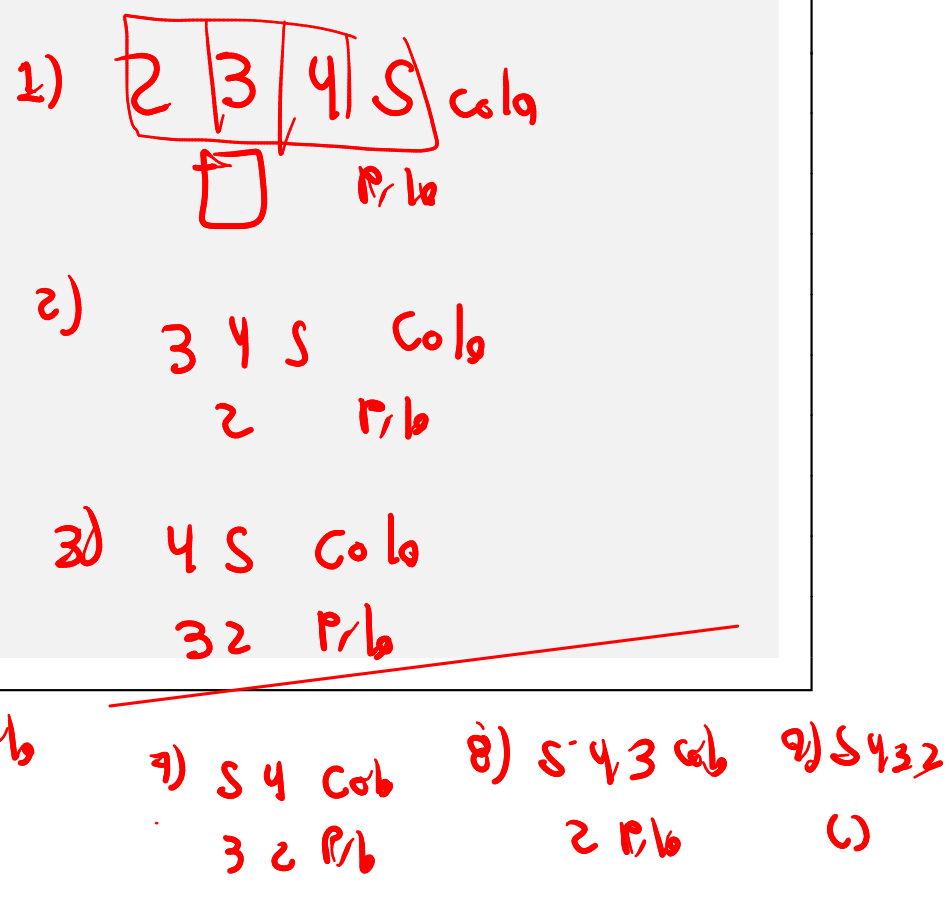
Lo que cuesta ahorrar un entero

Un ciclo que pregunte `tamano()` en cada vuelta, como el que imprime la cola dándole la vuelta, pasa de $\Theta(n)$ a $\Theta(n^2)$. Con el contador, la cola gasta un entero y responde en $\Theta(1)$.

(c) Invertir una cola con una pila I

La pila devuelve al revés lo que recibe. Si se le pasa la cola en orden y después se vuelca en la cola, sale invertida.

```
void invertir(Cola &c) {
  Pila p;
  while (!c.vacia()) {
    p.apilar(c.frente());
    c.desencolar();
  }
  while (!p.vacia()) {
    c.encolar(p.tope());
    p.desapilar();
  }
}
```



(c) Invertir una cola con una pila II

Con $\langle 3, 1, 4, 1, 5 \rangle$

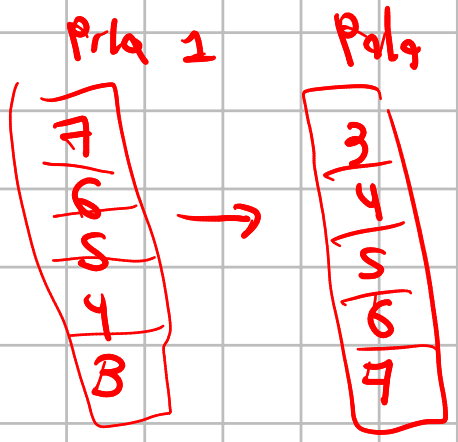
Momento	Cola, del frente al final	Pila, del tope al fondo
al empezar	$\langle 3, 1, 4, 1, 5 \rangle$	vacía
tras el primer ciclo	vacía	$\langle 5, 1, 4, 1, 3 \rangle$
tras el segundo ciclo	$\langle 5, 1, 4, 1, 3 \rangle$	vacía

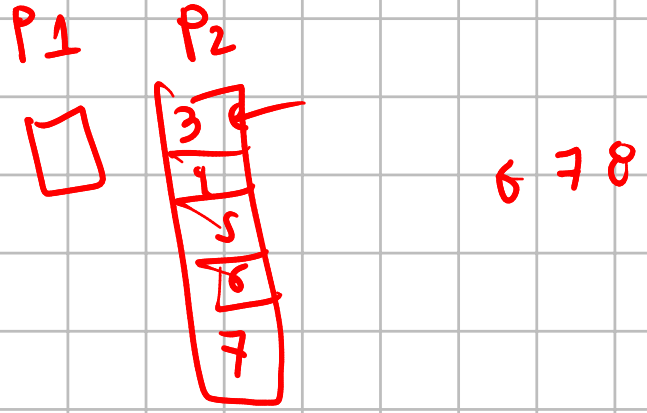
El programa imprime `antes: 3 1 4 1 5` y `despues: 5 1 4 1 3`.

	Costo	Por qué
tiempo	$\Theta(n)$	cada elemento sale y entra una vez en cada TAD, a $\Theta(1)$
espacio aparte	$\Theta(n)$	la pila guarda los n elementos a la vez

Una cola con dos pilas

Insertar 3 4 5 6 7, cuando pida el tope entonces devolver 3







El tope me lo da la P2 (invierte la P1), yo guardo los elementos de la P1 hasta que la p2 queda vacia

Una vez que quede vacia entonces paso los elementos de la p1 a la p2

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

¿En qué casilla queda el 5? I

Pregunta

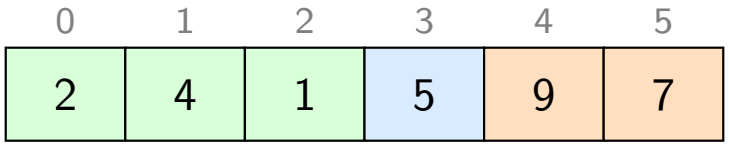
En $\langle 7, 2, 9, 4, 1, 5 \rangle$, ¿en qué casilla queda el 5 cuando el arreglo esté ordenado? ¿Hace falta ordenarlo para saberlo?

El 5 tiene una caracteristica, los que estan antes que el son menores o iguales
Y los que estan despues de el son mayores o iguales

No es necesario ordenar

¿En qué casilla queda el 5? II

Se cuentan los valores menores o iguales que 5, sin contarlo a él: son tres, el 2, el 4 y el 1. Con el arreglo ordenado el 5 tiene tres valores a su izquierda, así que queda en la casilla 3.



No hace falta ordenar. Basta separar: los menores o iguales a la izquierda, en cualquier orden, después el 5 y al final los mayores. Con eso el 5 ya está en la casilla que ocupará en el arreglo ordenado.

Pagar al partir I

Merge sort parte por la mitad sin mirar los datos y paga al final: mezclar dos mitades ordenadas las recorre enteras. Si al partir ya quedan a la izquierda los menores o iguales y a la derecha los mayores, ordenar cada lado deja todo el arreglo ordenado y combinar no cuesta nada.

Quicksort sobre $a[ini..fin]$ (CLRS, §7.1)

- **Dividir**: escoger un pivote x y reordenar para que queden primero los menores o iguales que x, después x y al final los mayores. El pivote queda en una posición q.
- **Conquistar**: ordenar $a[ini..q-1]$ y $a[q+1..fin]$ con el mismo procedimiento.
- **Combinar**: no hay nada que hacer.

¿Entendimos el problema? I

Partir es el paso que hace todo el trabajo. Se llama `particionar`.

Entradas

El arreglo a y dos índices `ini` $\leq$ `fin`. El pivote es el último del rango: $x = a[fin]$.

Salida

Una posición q y el arreglo reordenado: de `ini` a $q-1$ los menores o iguales que x, en q el pivote y de $q+1$ a `fin` los mayores. Dentro de cada lado no se pide orden.

Con $\langle 7, 2, 9, 4, 1, 5 \rangle$, $ini = 0$, $fin = 5$

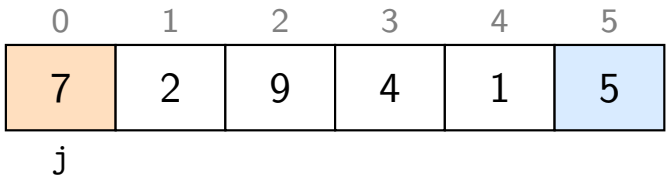
$x = 5$ y $q = 3$. Sirven $\langle 2, 4, 1, 5, 9, 7 \rangle$ y también $\langle 1, 4, 2, 5, 7, 9 \rangle$: lo que no puede pasar es que un mayor que 5 quede a la izquierda del pivote.

Primera idea: dos colas I

Con las colas de nodos ya construidas, partir es repartir.

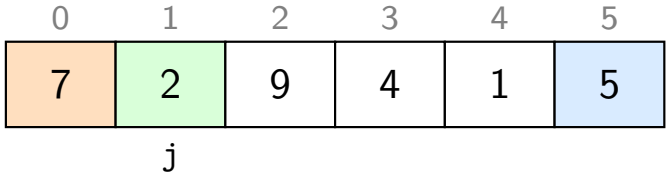
- Recorrer $a[ini..fin-1]$ y encolar cada valor en `menores` si es $\leq x$, o en `mayores` si no.
- Reescribir el arreglo desde `ini`: primero lo que sale de `menores`, después x y al final lo que sale de `mayores`. El pivote queda en $q = ini + \texttt{menores.tamano()}$.

Primera idea: dos colas II





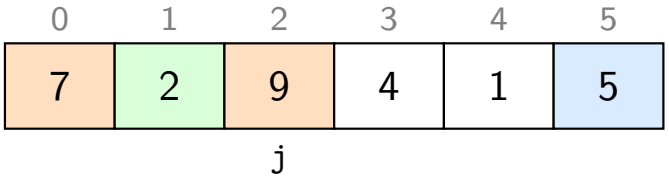
Llega el 7, mayor que 5: va a `mayores`.

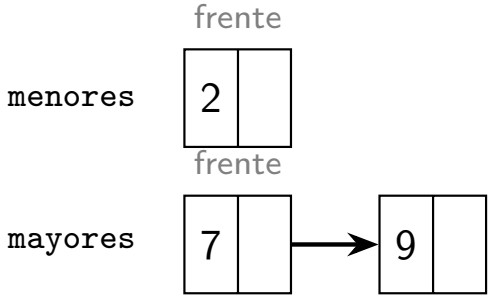




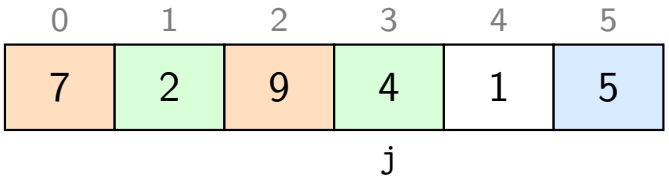
Llega el 2, menor o igual: va a `menores`.

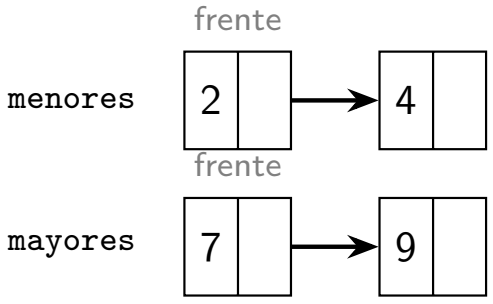
Primera idea: dos colas III





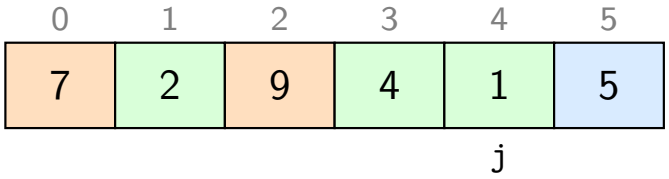
Llega el 9: va a `mayores`, detrás del 7.

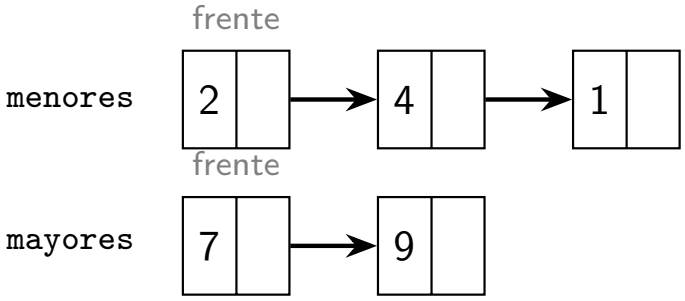




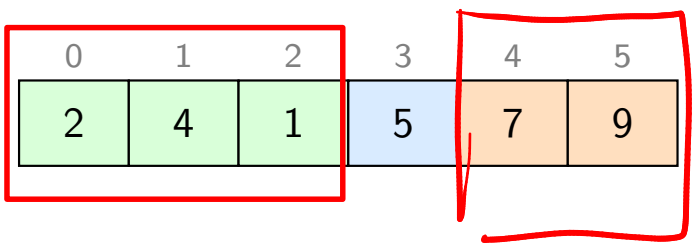
Llega el 4: va a `menores`.

Primera idea: dos colas IV





Llega el 1: va a `menores`. `j` llegó a `fin`.



`menores` vacía

`mayores` vacía

Se reescribe desde `ini`: se vacía `menores`, va el pivote y se vacía `mayores`. $q = 3$.

Primera idea: dos colas V

	Costo con $n = fin - ini + 1$	Por qué
tiempo	$\Theta(n)$	cada valor se encola y se desencola una vez
espacio aparte	$\Theta(n)$	un nodo por valor, repartidos en las dos colas

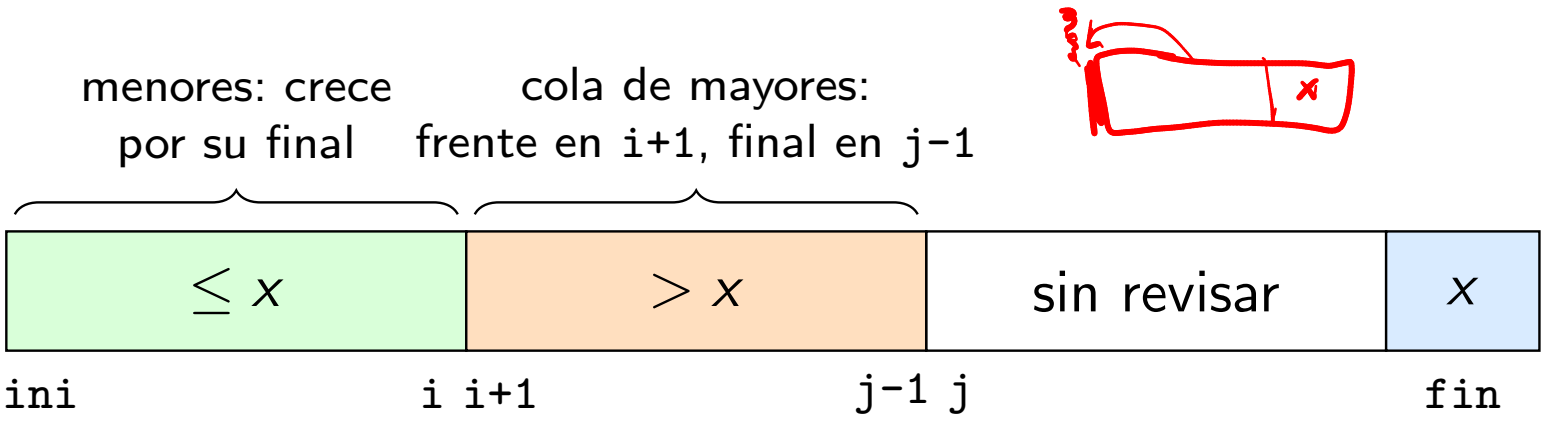
El tiempo no se puede bajar: hay que mirar cada valor. El espacio sí molesta, porque los valores ya caben en el arreglo.

Pregunta

¿Se puede partir sin pedir memoria, moviendo los valores dentro del mismo arreglo?

Las dos colas dentro del arreglo I

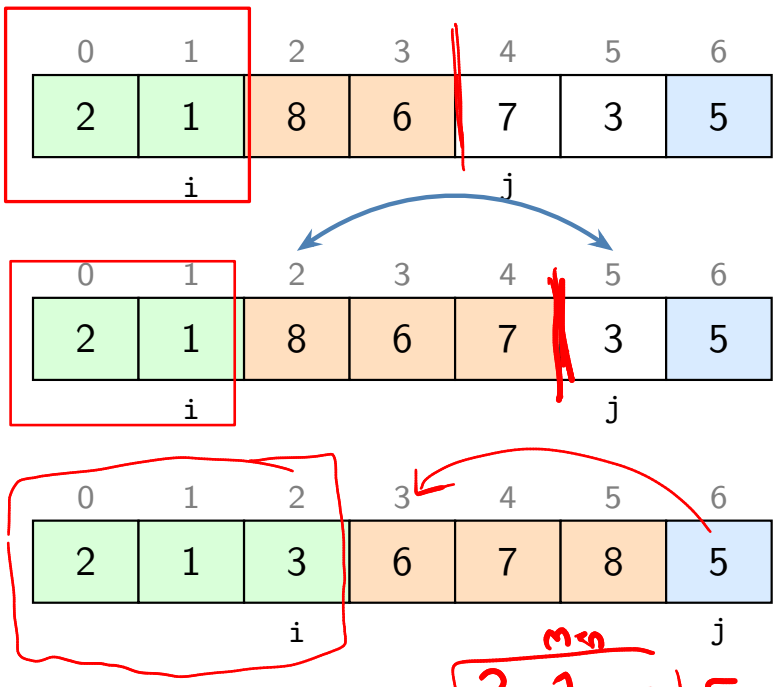
Sí se puede. Las dos colas caben en el mismo arreglo, una al lado de la otra, y dos índices marcan dónde termina cada una.



- `j` recorre de izquierda a derecha y revisa un valor por vuelta.
- Llega un mayor que x: ya está pegado al final de la cola de mayores. Entra solo con avanzar `j`.
- Llega un menor o igual: el frente de los mayores, $a[i+1]$, sale y pasa al final, a la casilla `j`; el menor ocupa su puesto, que ahora es el final de los menores. Ese movimiento es `intercambiar(a, i + 1, j)`.

Las dos colas dentro del arreglo II

Con $x = 5$ en $\langle 2, 1, 8, 6, 7, 3, 5 \rangle$, ya revisados el 2, el 1, el 8 y el 6:



llega el 7, mayor: avanza `j`

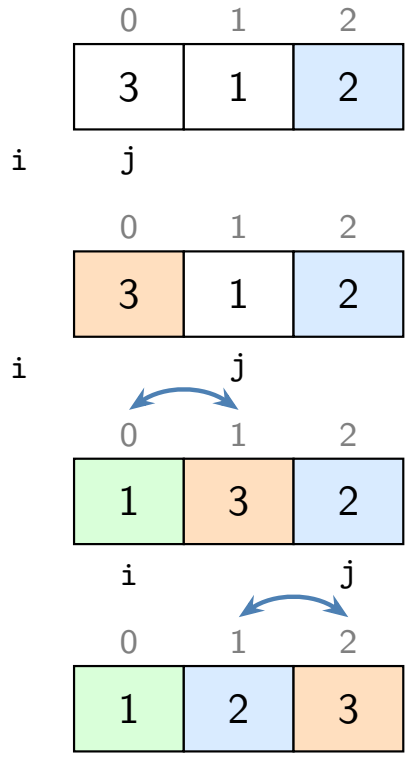
llega el 3, menor: sale el 8 del frente y el 3 toma su puesto

los mayores quedan 6, 7, 8

Al terminar, el pivote entra como un menor más: `intercambiar(a, i + 1, fin)` lo pone en el puesto del frente de los mayores, y ese frente pasa a la última casilla.

El ejemplo más pequeño: $\langle 3, 1, 2 \rangle$ I

$ini = 0$, $fin = 2$ y el pivote es $x = 2$.



Al empezar las dos colas están vacías: $i = -1$.

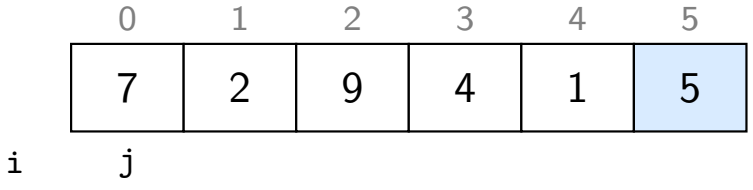
Llega el 3, mayor que 2: entra a la cola de mayores y solo avanza `j`.

Llega el 1: el 3 sale del frente y pasa al final; el 1 toma su puesto. `intercambiar(a, 0, 1)`.

`j` llegó a `fin`: el pivote entra como un menor. `intercambiar(a, 1, 2)` y $q = 1$.

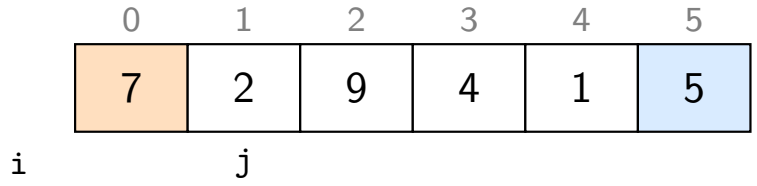
Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ I

$ini = 0$, $fin = 5$, $x = 5$. Verde: menores o iguales; naranja: la cola de mayores; blanco: sin revisar; celeste: el pivote. Cada fila de la tabla es una vuelta del ciclo.



i	j	$a[j]$	≤ 5	Qué pasa

Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ II

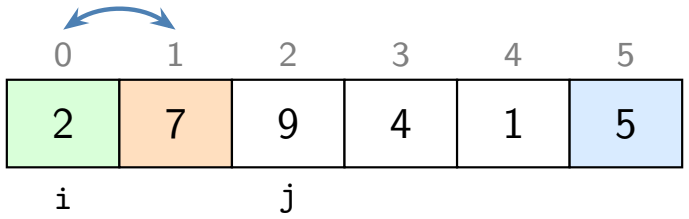


i	j	$a[j]$	≤ 5	Qué pasa
-1	0	7	no	el 7 entra al final de los mayores

Pregunta

Llega el 2, menor o igual que 5. ¿Quién sale del frente de los mayores y a qué casilla va?

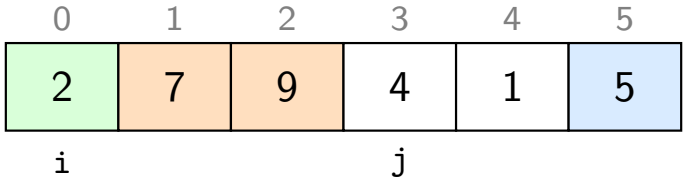
Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ III



Sale el 7, el único de la cola, y pasa a la casilla 1; el 2 queda en la 0, al final de los menores.

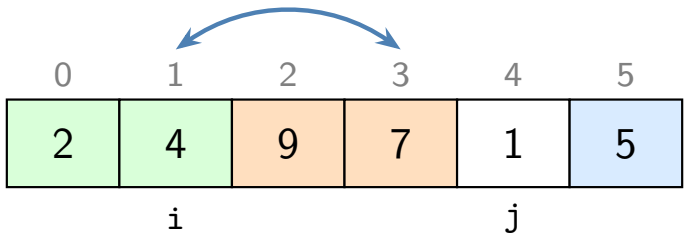
`i`	`j`	$a[j]$	≤ 5	Qué pasa
-1	0	7	no	el 7 entra al final de los mayores
-1	1	2	sí	sale el 7: `intercambiar(a, 0, 1)`

Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ IV



i	j	$a[j]$	≤ 5	Qué pasa
-1	0	7	no	el 7 entra al final de los mayores
-1	1	2	sí	sale el 7: `intercambiar(a, 0, 1)`
0	2	9	no	el 9 entra al final de los mayores

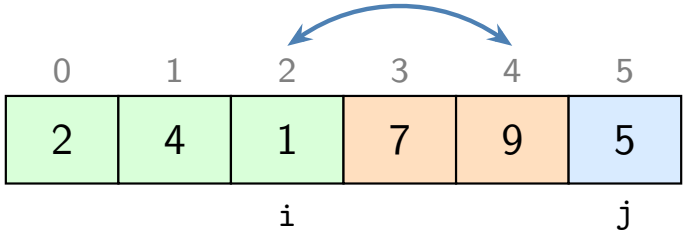
Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ V



La cola de mayores era 7, 9: el 7 salió del frente y quedó detrás del 9.

`i`	`j`	$a[j]$	≤ 5	Qué pasa
-1	0	7	no	el 7 entra al final de los mayores
-1	1	2	sí	sale el 7: `intercambiar(a, 0, 1)`
0	2	9	no	el 9 entra al final de los mayores
0	3	4	sí	sale el 7: `intercambiar(a, 1, 3)`

Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ VI

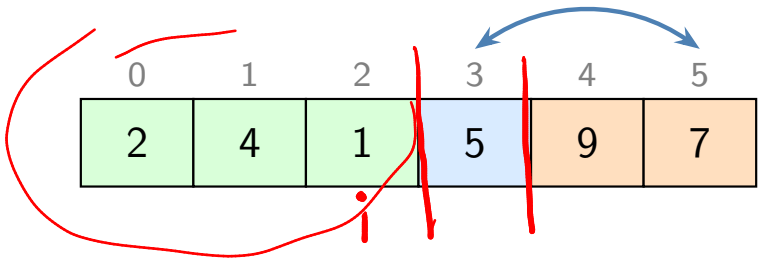


i	j	$a[j]$	≤ 5	Qué pasa
-1	0	7	no	el 7 entra al final de los mayores
-1	1	2	sí	sale el 7: `intercambiar(a, 0, 1)`
0	2	9	no	el 9 entra al final de los mayores
0	3	4	sí	sale el 7: `intercambiar(a, 1, 3)`
1	4	1	sí	sale el 9: `intercambiar(a, 2, 4)`

Pregunta

`j` llegó a `fin` y no queda nada sin revisar. ¿En qué casilla va el pivote y a dónde va el valor que estaba ahí?

Paso a paso con $\langle 7, 2, 9, 4, 1, 5 \rangle$ VII



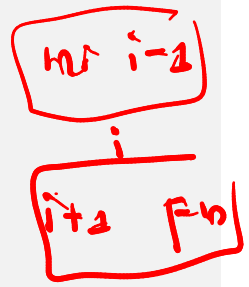
En la casilla $i + 1 = 3$, el frente de los mayores. El 7 sale y pasa a la casilla 5: el pivote entró como un menor más.

i	j	$a[j]$	≤ 5	Qué pasa
-1	0	7	no	el 7 entra al final de los mayores
-1	1	2	sí	sale el 7: `intercambiar(a, 0, 1)`
0	2	9	no	el 9 entra al final de los mayores
0	3	4	sí	sale el 7: `intercambiar(a, 1, 3)`
1	4	1	sí	sale el 9: `intercambiar(a, 2, 4)`
2	5			$j =$ *fin*: `intercambiar(a, 3, 5)`, $q = 3$

Los dos lados no quedan ordenados, pero el 5 ya está en la casilla que ocupará al final.

El diseño en pseudocódigo I

```
particionar(a, ini, fin):
  x <- a[fin]
  i <- ini - 1                  // final de los menores
  para j desde ini hasta fin - 1:
    si a[j] <= x:
      i <- i + 1
      intercambiar a[i] y a[j]    // sale el frente
  intercambiar a[i + 1] y a[fin]  // entra el pivote
  devolver i + 1
```



Un mayor no se mueve: entra a su cola cuando avanza `j`. El pseudocódigo hace un intercambio por cada menor o igual, y uno más al final para el pivote.

particionar en C++ (CLRS, §7.1) I

```
void intercambiar(Elemento a[], int p, int q) {
  Elemento temp = a[p];
  a[p] = a[q];
  a[q] = temp;
}
```

`intercambiar` hace tres asignaciones y usa una variable: cuesta $\Theta(1)$ en tiempo y en espacio.
`particionar`, en la página siguiente, es el pseudocódigo línea por línea, con el `para` escrito como `while`. Las colas de la primera idea ya no aparecen como objetos: viven en el arreglo, entre `ini` e `i` y entre `i+1` y `j-1`.

particionar en C++ (CLRS, §7.1) II

```
// Parte a[ini..fin] alrededor de x = a[fin] y
// devuelve la posicion final del pivote. Exige ini <= fin.
int particionar(Elemento a[], int ini, int fin) {
  Elemento x = a[fin];
  int i = ini - 1;
  int j = ini;
  while (j < fin) {
    if (a[j] <= x) {
      i = i + 1;
      intercambiar(a, i, j);
    }
    j = j + 1;
  }
  intercambiar(a, i + 1, fin);
  return i + 1;
}
```

quicksort I

```
void quicksort(Elemento a[], int ini, int fin) {
  if (ini < fin) {
    int q = particionar(a, ini, fin);
    quicksort(a, ini, q - 1);
    quicksort(a, q + 1, fin);
  }
}
```

La llamada inicial es `quicksort(a, 0, n - 1)`. Un rango de cero o un elemento no entra al `if`: ya está ordenado.

Sobre $\langle 6, 2, 9, 4, 1, 7, 3, 8 \rangle$

```
quicksort:   1 2 3 4 6 7 8 9
```

El árbol de llamadas sobre $\langle 7, 2, 9, 4, 1, 5 \rangle$ I

Nivel 0: una llamada. El 5 queda fijo en la casilla 3.

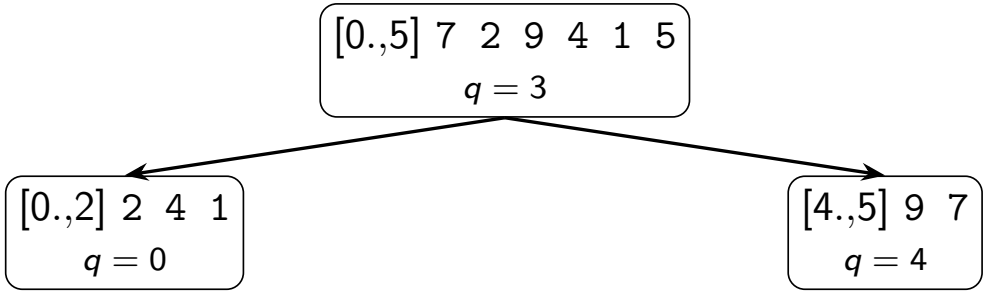
[0.,5] 7 2 9 4 1 5
$q = 3$

arreglo:

0	1	2	3	4	5
2	4	1	5	9	7

El árbol de llamadas sobre $\langle 7, 2, 9, 4, 1, 5 \rangle$ II

Nivel 1: dos llamadas, una por lado. Quedan fijos el 1 y el 7.

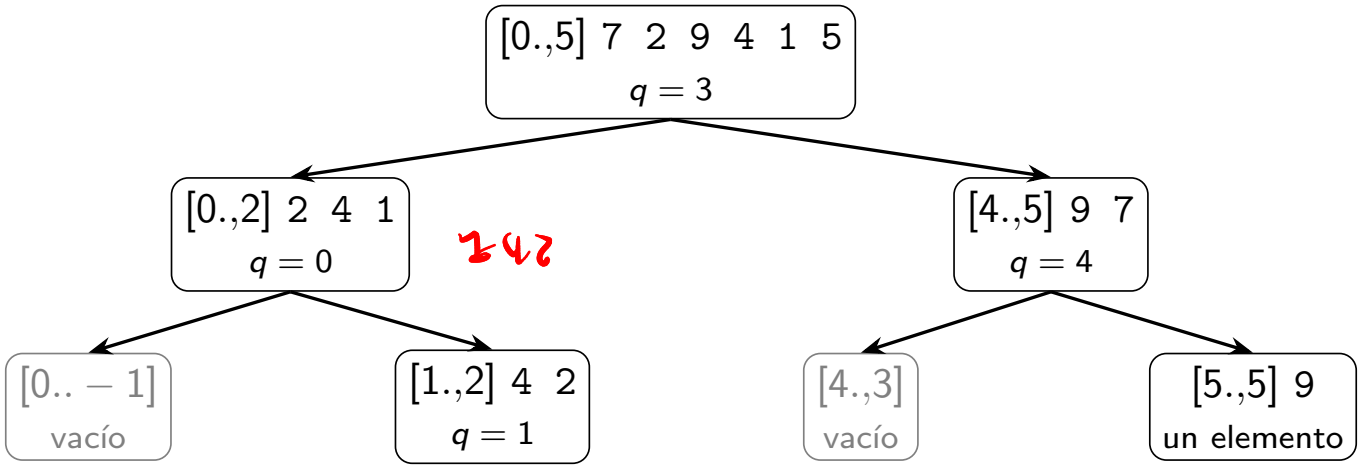


arreglo:

0	1	2	3	4	5
1	4	2	5	7	9

El árbol de llamadas sobre $\langle 7, 2, 9, 4, 1, 5 \rangle$ III

Nivel 2: los rangos vacíos y el de un elemento no entran al `if`; el 2 queda fijo.

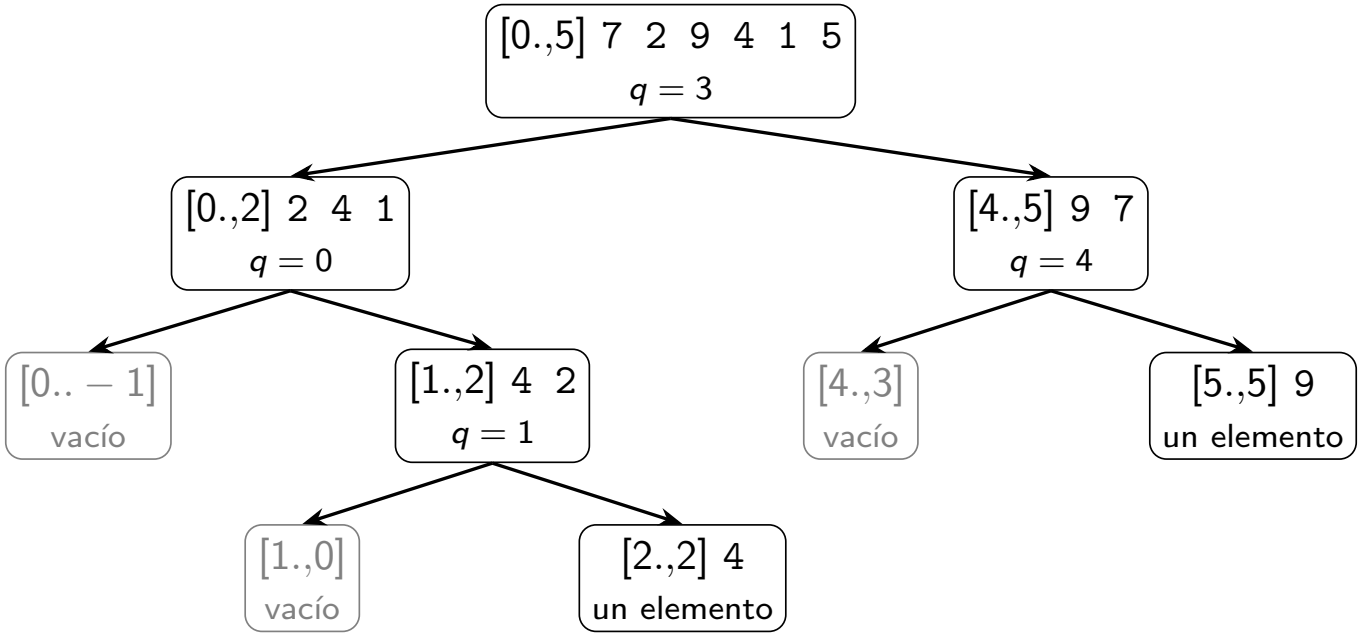


arreglo:

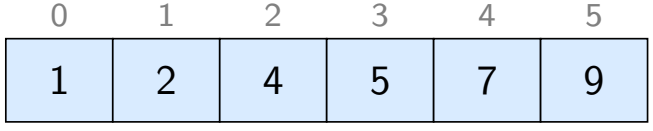
0	1	2	3	4	5
1	2	4	5	7	9

El árbol de llamadas sobre $\langle 7, 2, 9, 4, 1, 5 \rangle$ IV

Nivel 3: solo queda el 4, solo en su casilla. Todo el arreglo está ordenado.



arreglo:



Las llamadas pendientes forman una pila I

Mientras se ordena la izquierda, la derecha espera como un rango $[ini, fin]$, y el último que entra es el primero que se atiende: una pila. Cada fila saca un rango; si tiene dos o más elementos, lo parte y apila los dos lados, el izquierdo encima.

Saca	q	Arreglo	Pila después (tope a la izquierda)
		7 2 9 4 1 5	$[0,5]$
$[0,5]$	3	2 4 1 5 9 7	$[0,2]$ $[4,5]$
$[0,2]$	0	1 4 2 5 9 7	$[0,-1]$ $[1,2]$ $[4,5]$
$[0,-1]$	vacío		$[1,2]$ $[4,5]$
$[1,2]$	1	1 2 4 5 9 7	$[1,0]$ $[2,2]$ $[4,5]$
$[1,0]$	vacío		$[2,2]$ $[4,5]$
$[2,2]$	uno		$[4,5]$
$[4,5]$	4	1 2 4 5 7 9	$[4,3]$ $[5,5]$
$[4,3]$	vacío		$[5,5]$
$[5,5]$	uno		vacía

Los rangos salen en el orden de las llamadas de `quicksort`, y la pila nunca guarda más de tres.

quicksortPila: la misma pila, escrita a mano I

```
void quicksortPila(Elemento a[], int ini, int fin) {
  Pila pendientes;
  pendientes.apilar(fin);
  pendientes.apilar(ini);
  while (!pendientes.vacia()) {
    int p = pendientes.tope();
    pendientes.desapilar();
    int r = pendientes.tope();
    pendientes.desapilar();
    if (p < r) {
      int q = particionar(a, p, r);
      pendientes.apilar(r);
      pendientes.apilar(q + 1);
      pendientes.apilar(q - 1);
      pendientes.apilar(p);
    }
  }
}
```

`quicksortPila`: la misma pila, escrita a mano II

Usa la pila de nodos de la sesión (`#include "pila_nodos.h"`) y guarda cada rango como dos enteros: se apila `fin` y después `ini`, así que `ini` sale primero. El `if` es el mismo de `quicksort`; las dos llamadas recursivas se cambiaron por dos rangos apilados. El lado derecho se apila primero para que el izquierdo quede encima y se atienda antes, como en la versión recursiva.

Sobre los mismos arreglos

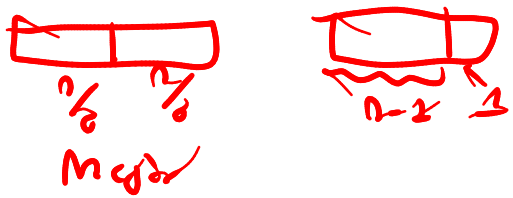
```
quicksortPila: 1 23 4 5 78 9
quicksortPila: 1 2 3 4 6 7 8 9
```

La diferencia está en dónde vive la pila: los nodos se piden con `new` y no ocupan registros en la pila de llamadas del programa.

Lo que cuesta cada pieza I

Pieza	Tiempo	Espacio aparte	Por qué
`intercambiar`	$\Theta(1)$	$\Theta(1)$	tres asignaciones y `temp`
partir con dos colas	$\Theta(n)$	$\Theta(n)$	cada valor entra y sale de una cola; un nodo por valor
`particionar`	$\Theta(n)$	$\Theta(1)$	una vuelta por casilla; las colas viven en el arreglo
`apilar`, `desapilar`	$\Theta(1)$	un nodo	se trabaja solo en la cima
`quicksort`	$\Theta(n \log n)$ a $\Theta(n^2)$	$\Theta(\log n)$ a $\Theta(n)$	depende de dónde caiga q; el espacio es la pila de llamadas
`quicksortPila`	$\Theta(n \log n)$ a $\Theta(n^2)$	$\Theta(\log n)$ a $\Theta(n)$	las mismas particiones; el espacio son los rangos pendientes

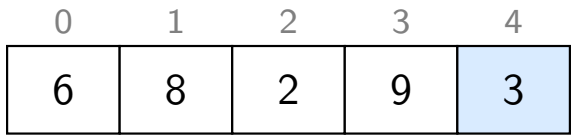
Los dos extremos de `quicksort` salen de dónde cae el pivote, y la sección siguiente los calcula. La profundidad de la pila, de llamadas o de rangos, es la de *La pila de llamadas*.



Partir otros arreglos I

Pregunta

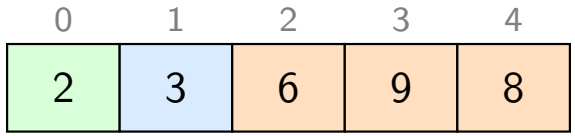
`particionar` sobre $\langle 6, 8, 2, 9, 3 \rangle$ con $ini = 0$ y $fin = 4$: ¿qué q devuelve y cómo queda el arreglo?



Partir otros arreglos II

$\langle 6, 8, 2, 9, 3 \rangle$, $x = 3$

$q = 1$ y queda $\langle 2, 3, 6, 9, 8 \rangle$.

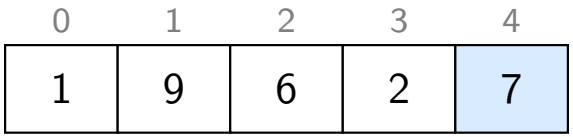


Solo el 2 es menor o igual que 3. La cola de mayores pasa por 6, 8; luego 8, 6; luego 8, 6, 9. Al final el pivote saca al 8 del frente.

Partir otros arreglos III

Pregunta

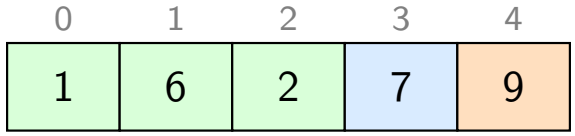
`particionar` sobre $\langle 1, 9, 6, 2, 7 \rangle$ con $ini = 0$ y $fin = 4$: ¿qué q devuelve y cómo queda el arreglo?



Partir otros arreglos IV

$\langle 1, 9, 6, 2, 7 \rangle$, $x = 7$

$q = 3$ y queda $\langle 1, 6, 2, 7, 9 \rangle$.

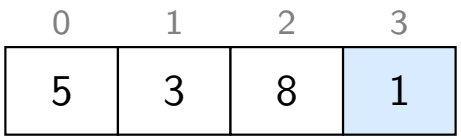


El 1 llega con la cola de mayores vacía y se intercambia consigo mismo. Después el 9 sale del frente dos veces: con el 6 y con el 2.

Partir otros arreglos V

Pregunta

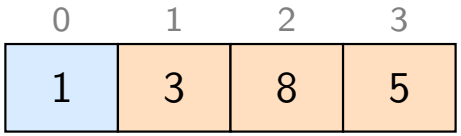
`particionar` sobre $\langle 5, 3, 8, 1 \rangle$ con $ini = 0$ y $fin = 3$: ¿qué q devuelve y cómo queda el arreglo?



Partir otros arreglos VI

$\langle 5, 3, 8, 1 \rangle$, $x = 1$

$q = 0$ y queda $\langle 1, 3, 8, 5 \rangle$.



Nadie es menor o igual que 1: `i` se queda en -1 y el pivote toma la casilla 0. Los mayores 5, 3, 8 quedan 3, 8, 5.

Por qué queda ordenado I

En tres frases

Cada llamada a `particionar` deja su pivote en la casilla que ocupará en el arreglo ordenado. Las dos llamadas que siguen trabajan cada una de su lado y nunca se cruzan: nada de la izquierda tiene que pasar a la derecha. Cuando los rangos llegan a cero o un elemento, cada valor ya fue pivote o quedó solo en su casilla.

En el árbol de $\langle 7, 2, 9, 4, 1, 5 \rangle$ se ve igual: cada nivel pinta de celeste los pivotes que fija, y ninguno se vuelve a mover.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

El costo de `particionar` I

El ciclo da exactamente $fin - ini$ vueltas, con una comparación $a[j] \leq x$ y a lo sumo un intercambio en cada una.

	Costo con $n = fin - ini + 1$	Por qué
comparaciones	$n - 1$	una por vuelta
tiempo	$\Theta(n)$	cada vuelta es $\Theta(1)$
espacio aparte	$\Theta(1)$	`x`, `i`, `j` y `temp`

El costo de quicksort entonces lo decide un solo dato: dónde queda q, porque eso fija el tamaño de las dos llamadas.

El caso balanceado I

Si el pivote cae siempre cerca de la mitad, cada llamada se parte en dos de tamaño $n/2$:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1)$$

Es la recurrencia de merge sort: el árbol de llamadas tiene $\log_2 n$ niveles y cada nivel hace $\Theta(n)$ de trabajo en total.

$$T(n) = \Theta(n \log n)$$

No hace falta la mitad exacta

Si cada partición dejara un décimo de un lado y nueve décimos del otro, el árbol tendría $\log_{10/9} n$ niveles, que sigue siendo $\Theta(\log n)$, y cada nivel costaría a lo sumo $\Theta(n)$: el total sigue en $\Theta(n \log n)$ (CLRS, §7.2).

Pregunta I

¿Qué pasa si el arreglo ya viene ordenado?

Siga `particionar` sobre $\langle 1, 2, 3, 4, 5 \rangle$. ¿Dónde queda el pivote? ¿De qué tamaño son las dos llamadas que siguen?

El peor caso: ya ordenado I

El pivote es el 5, el mayor. Todos los $a[j]$ cumplen $a[j] \leq 5$, así que `i` avanza con `j` y cada intercambio es de una casilla consigo misma. Queda $q = 4$: la izquierda tiene cuatro elementos, todavía ordenados, y la derecha ninguno.

Llamada	Elementos	q	Comparaciones
$a[0.,4]$	5	4	4
$a[0.,3]$	4	3	3
$a[0.,2]$	3	2	2
$a[0.,1]$	2	1	1
			10

Cada llamada quita un solo elemento: el pivote.

El peor caso: ya ordenado II

Teorema

Sobre un arreglo de $n \geq 1$ elementos distintos en orden creciente, quicksort hace $C(n) = n(n-1)/2$ comparaciones, y $C(n) \in \Theta(n^2)$.

Demostración. Se procede de forma directa: primero se plantea la recurrencia de C y se resuelve, y después se acota con testigos.
Desarrollo. En un rango ordenado de $n \geq 2$ elementos el pivote es el máximo, `particionar` hace $n-1$ comparaciones, devuelve $q = \mathit{fin}$ y deja la izquierda igual, ordenada, con $n-1$ elementos. La derecha está vacía. Entonces

$$C(n) = (n-1) + C(n-1), \qquad C(1) = 0,$$

y desplegando, $C(n) = (n-1) + (n-2) + \cdots + 1 + 0 = \dfrac{n(n-1)}{2}$.

El peor caso: ya ordenado III

Cota superior: para $n \geq 1$,

$$\frac{n(n-1)}{2} \leq \frac{n^2}{2},$$

así que $C(n) \leq c\, n^2$ con $c = 1/2$ y $k = 1$.
Cota inferior: para $n \geq 2$ se tiene $n^2 \geq 2n$, es decir, $2n^2 - 2n \geq n^2$, y dividiendo entre 4,

$$\frac{n(n-1)}{2} \geq \frac{n^2}{4},$$

así que $C(n) \geq c\, n^2$ con $c = 1/4$ y $k = 2$.

Conclusión

Por lo tanto, se puede concluir que $C(n) \in O(n^2)$ con testigos $c = 1/2$ y $k = 1$, y $C(n) \in \Omega(n^2)$ con testigos $c = 1/4$ y $k = 2$: $C(n) \in \Theta(n^2)$.

El peor caso: ya ordenado IV

La entrada más fácil para insertion sort

Un arreglo ya ordenado le cuesta a insertion sort $n - 1$ comparaciones y a quicksort $n(n-1)/2$. Con el pivote en el último lugar, ordenar algo que ya está ordenado es el peor caso. Al revés pasa lo mismo: el pivote es el mínimo y cada llamada vuelve a quitar uno solo.

La pila de llamadas I

Quicksort no pide arreglo auxiliar, pero cada llamada pendiente ocupa un registro en la pila de llamadas, con `a`, `ini`, `fin` y `q`. El espacio aparte es la profundidad máxima de la recursión.

Caso	Profundidad	Por qué
balanceado	$\Theta(\log n)$	cada nivel parte el rango por la mitad
ya ordenado	$\Theta(n)$	cada nivel quita un elemento
`quicksortPila`	la misma	los rangos van en nodos pedidos con `new`

Lo que se ve al correrlo

Sobre un arreglo ya ordenado queda una llamada pendiente por elemento. Con la pila de llamadas de 8 MB que da Linux por omisión, el programa ordena 100 000 elementos y se cae con 300 000, por desbordar la pila antes de terminar.

Escoger otro pivote I

El peor caso aparece porque $a[fin]$ es el máximo o el mínimo del rango. El pivote no tiene que ser el último: basta escoger otra casilla e intercambiarla con $a[fin]$ antes de partir, y `particionar` queda igual.

- **El del medio**: intercambiar $a[(ini + fin)/2]$ con $a[fin]$. Sobre un arreglo ya ordenado el pivote es la mediana y las particiones quedan balanceadas. Existen otras entradas que lo hacen cuadrático, pero no son las que aparecen solas.
- **Al azar**: escoger la casilla con un número aleatorio entre ini y fin. Ninguna entrada fija provoca siempre el peor caso, y el tiempo esperado es $\Theta(n \log n)$ sobre cualquier entrada (CLRS, §7.3 y §7.4). El peor caso sigue siendo $\Theta(n^2)$, pero depende de la suerte y no de los datos.

Los tres algoritmos sobre la misma entrada I

Comparaciones contadas con un mismo programa: insertion sort y quicksort sobre el arreglo, merge sort sobre la lista enlazada. El mezclado toma valores entre 0 y 10 006 sin orden.

Entrada	n	insertion sort	merge sort	quicksort
mezclado	16	78	48	59
	128	4 166	740	811
	1024	268 112	8 584	11 414
	4096	4 162 689	43 220	54 266
ordenado	16	15	32	120
	128	127	448	8 128
	1024	1 023	5 120	523 776
	4096	4 095	24 576	8 386 560
al revés	16	135	32	120
	128	8 255	448	8 128
	1024	524 799	5 120	523 776
	4096	8 390 655	24 576	8 386 560

Los tres algoritmos sobre la misma entrada II

- Sobre el mezclado, quicksort hace 54 266 comparaciones con 4096 elementos, 1,26 veces las de merge sort, y unas 77 veces menos que insertion sort.
- Sobre el ordenado los papeles se invierten: insertion sort hace 4 095 y quicksort 8 386 560, que es $4096 \cdot 4095/2$, la cifra del teorema.
- Merge sort no cambia de orden de magnitud con la entrada: parte sin mirar los datos.

Los tres algoritmos sobre la misma entrada III

Lo que la tabla no cuenta

Quicksort hace más comparaciones que merge sort sobre el mezclado, y aun así CLRS (cap. 7) lo presenta como la opción práctica más común para ordenar: sobre un arreglo no reserva nodos, no sigue punteros y recorre casillas vecinas. Contar comparaciones compara algoritmos, no programas.

Los tres, lado a lado I

Algoritmo	Mejor	Peor	Espacio aparte	En el sitio	Por qué
insertion, arreglo	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$	sí	corre lo anterior; si ya está ordenado, nada
merge, lista	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(\log n)$	sí	parte por la mitad sin mirar los datos
merge, arreglo	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	no	mezcla en un arreglo auxiliar
quicksort, arreglo	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$ a $\Theta(n)$	sí	las partes dependen del pivote

En merge sort sobre la lista y en quicksort, el espacio aparte es la pila de llamadas. Sobre la lista, merge sort no copia datos: empalma nodos.

Los tres, lado a lado II

Cuál escoger

- Pocos datos, o datos casi ordenados: insertion sort.
- Datos en una lista enlazada, o un peor caso garantizado: merge sort.
- Un arreglo sin orden previo y sin memoria de sobra: quicksort con un pivote que no sea siempre el último.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Para el juez: UVa 10041 — Vito's Family I

UVa 10041 — Vito's Family,
`https://onlinejudge.org/external/100/10041.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=982`

Qué pide

Vito tiene r parientes en una misma avenida, con $0 < r < 500$, y cada uno vive en un número s_i con $0 < s_i < 30\,000$. Hay que escoger el número de su casa para que la suma de las distancias $|s_i - s_j|$ a todos sea mínima, e imprimir esa suma. La primera línea trae el número de casos.

Entradas y salidas del ejemplo

Con $\langle 2, 4\rangle$ la respuesta es 2: cualquier casa entre 2 y 4 suma 2. Con $\langle 2, 4, 6\rangle$ es 4: la casa en el 4 suma $2 + 0 + 2$, y en el 3 sumaría $1 + 1 + 3 = 5$.

Para el juez: UVa 10041 — Vito's Family II

Cómo atacarlo con lo de hoy

La casa óptima es la mediana: si hay más parientes a la derecha que a la izquierda, correrse un paso a la derecha baja la suma. Se ordena con quicksort, se toma $s[r/2]$ y se suman las distancias. Cuesta $\Theta(r \log r)$ esperado y $\Theta(r^2)$ en el peor caso, que con $r < 500$ entra igual.

Las trampas

- Los números no vienen ordenados.
- El promedio no sirve: con $\langle 1, 1, 1, 1, 1, 1000 \rangle$ el promedio es 167,5 y la mediana en 1, que suma 999.
- Varios parientes pueden vivir en el mismo número, y la casa puede coincidir con la de uno de ellos.

Para el juez: UVa 10905 — Children's Game I

UVa 10905 — Children's Game,
`https://onlinejudge.org/external/109/10905.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1846`

Qué pide

Cada caso trae $N \leq 50$ enteros positivos. Hay que pegarlos uno detrás de otro, en algún orden, para formar el entero más grande posible, e imprimirlo. La entrada termina con $N = 0$.

Entradas y salidas del ejemplo

Con 123, 124, 56 y 90 la respuesta es 9056124123. Si se agrega un 9, es 99056124123: el 9 va antes que el 90. Con cinco nueves es 99999.

Para el juez: UVa 10905 — Children's Game II

Cómo atacarlo con lo de hoy

`particionar` solo pregunta si un elemento va antes que el pivote. Los números se leen como texto con `scanf("%s")`, y `vaAntes(p, q)` pega p con q y q con p y compara los dos resultados con `strcmp`. La condición $a[j] \leq x$ se cambia por `!vaAntes(x, a[j])`, y se ordena un arreglo de índices para no copiar cadenas al intercambiar. Con $N \leq 50$, aun el peor caso de quicksort son 1225 comparaciones por caso.

Las trampas

- Ordenar por valor no sirve: 90 es mayor que 9, pero 990 gana a 909.
- Ordenar el texto de mayor a menor tampoco: 121 va después de 12 en ese orden, y 12121 gana a 12112, así que el 12 va primero.
- El enunciado no acota los dígitos: el resultado no cabe en un entero y se arma como texto.

Contenido I

1 Las tres implementaciones enlazadas

2 Ejercicios sobre las implementaciones

3 Pagar al partir: quicksort

4 Lo que cuesta quicksort

5 Para el juez

6 Ejercicios

Ejercicio en parejas: todos iguales I

Ejecute `quicksort` sobre $\langle 4, 4, 4, 4, 4 \rangle$.

Las preguntas

1. ¿Qué devuelve la primera llamada a `particionar`?
2. ¿De qué tamaño son las dos llamadas que siguen?
3. ¿Cuántas comparaciones hace en total con n elementos iguales? ¿Ayuda tomar el pivote del medio?

Ejercicio en parejas: todos iguales II

Lo que debe salir

Todos cumplen $a[j] \leq 4$, así que `i` avanza con `j` y `particionar` devuelve $q = fin$: la izquierda tiene $n - 1$ elementos y la derecha ninguno. Es el mismo caso del arreglo ordenado, $n(n-1)/2$ comparaciones, $\Theta(n^2)$.

El pivote del medio no ayuda: todos los candidatos valen lo mismo.

El problema está en la condición $a[j] \leq x$, que manda a todos los iguales al mismo lado.

Ejercicio en parejas: solo un lado I

Se quiere el k-ésimo menor de un arreglo, contando desde 0, sin ordenarlo entero. Después de `particionar`, el pivote ya está en la casilla q que ocuparía con el arreglo ordenado.

Las preguntas

1. Si $k = q$, ¿cuál es la respuesta? ¿Y si $k < q$, o si $k > q$, en qué rango hay que seguir buscando?
2. Escriba `seleccionar(a, ini, fin, k)` con un `while`, sin recursión.
3. ¿Cuánto cuesta si cada partición deja la mitad? ¿Y en el peor caso?

Ejercicio en parejas: solo un lado II

Lo que debe salir

Si $k = q$, la respuesta es $a[q]$. Si $k < q$, el k-ésimo está en $a[ini..q-1]$, y si $k > q$ en $a[q+1..fin]$. A diferencia de quicksort, se sigue por un solo lado: el ciclo mueve `fin` a $q-1$ o `ini` a $q+1$ y vuelve a particionar, hasta que $q = k$.

Con particiones a la mitad el trabajo es $n + n/2 + n/4 + \cdots \leq 2n$: $\Theta(n)$, menos que ordenar. En el peor caso cada partición quita un solo elemento y vuelve a ser $\Theta(n^2)$. El espacio aparte es $\Theta(1)$, porque no hay recursión.

Propuestos para la casa I

1. **De mayor a menor.** ¿Qué línea de `particionar` se cambia para que quicksort ordene de mayor a menor? ¿Cómo quedan las zonas?
2. **La traza al revés.** Siga quicksort sobre $\langle 5, 4, 3, 2, 1 \rangle$. ¿Dónde queda cada pivote y cuántas comparaciones hace en total?
3. **Menos pila.** Cambie la segunda llamada recursiva por un `while` que siga con el rango más grande y llame recursivamente solo con el más pequeño. ¿Qué profundidad alcanza la pila en el peor caso?
4. **Quicksort sobre la lista enlazada.** `i` y `j` solo avanzan. ¿Se puede escribir `particionar` sobre una lista simple, con punteros en lugar de índices? ¿Qué pivote es barato de tomar?
5. **Tres zonas.** Escriba un `particionar` que deje los menores, después los iguales al pivote y al final los mayores, y que devuelva dónde empiezan y terminan los iguales.

Pistas para los propuestos I

1. Solo cambia la comparación: $a[j] \geq x$. Las zonas se voltean: a la izquierda quedan los mayores o iguales y a la derecha los menores.
2. El primer pivote es el 1, el mínimo: queda en la casilla 0 y la derecha tiene cuatro elementos, ya no en orden decreciente. Siga la traza: el total coincide con el del arreglo ordenado.
3. Siempre se llama con el rango pequeño, que tiene a lo sumo la mitad de los elementos. Piense cuántas veces se puede partir n por la mitad antes de llegar a 1.
4. Sí: los dos punteros avanzan de a un nodo, como `i` y `j`. El pivote barato es el primer nodo; los intercambios pueden cambiar datos sin tocar los enlaces.
5. Este queda abierto: cuatro zonas en vez de tres, con un índice más para marcar dónde empieza cada una.

Resumen I

- La cola doble necesita la lista doblemente enlazada con cabeza y `ultimo`: quitar el último exige llegar al penúltimo, y eso solo lo da `anterior`.
- Sin contador, `vacia` sigue en $\Theta(1)$ y `tamano` pasa a $\Theta(n)$. Invertir una cola con una pila cuesta $\Theta(n)$ en tiempo y en espacio.
- `particionar` deja el pivote en su casilla definitiva, con los menores o iguales a la izquierda y los mayores a la derecha, en $\Theta(n)$ y sin arreglo auxiliar: las dos colas de la primera idea caben dentro del arreglo.
- Las llamadas pendientes de quicksort son una pila de rangos; `quicksortPila` la escribe con la pila de nodos y gasta el mismo espacio, $\Theta(\log n)$ a $\Theta(n)$.

Resumen II

- Quicksort paga al partir y no paga al combinar: $\Theta(n \log n)$ si las particiones son balanceadas, $\Theta(n^2)$ si el pivote es siempre el máximo o el mínimo, como con el pivote en el último lugar sobre un arreglo ya ordenado.

Lo que sigue

Los contenedores `list`, `deque`, `stack` y `queue` de la biblioteca: qué implementación tiene cada uno por dentro y qué cuestan sus operaciones, comparados con los TAD construidos a mano.

Referencias I


T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 7, quicksort: §7.1 descripción y `PARTITION`, §7.2 desempeño, §7.3 versión aleatoria, §7.4 análisis.


R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 14, búsqueda y ordenamiento.


N. Kalicharan. *Data Structures in C*. 2008. Capítulo 4, pilas y colas.